LAPRAK TEKNOLOGI IOT II

PRATIKUM 4 DAN 5

Oleh:

NAMA : M. Kurnia Akbar

NPM : 25782101

KELAS : TRI – 3D

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

Andi Saptono, S.Kom

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2025/2026

4.5.1 Praktikum 1: *Real-Time Dashboard* & Sinkronisasi Tombol Fisik

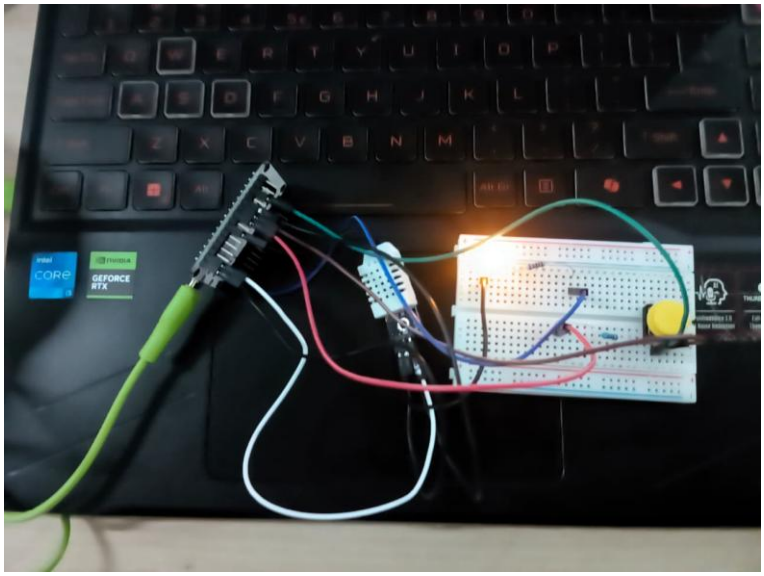
Skenario: Anda memiliki lampu LED (simulasi aktuator/lampu cerdas) yang bisa dihidupkan melalui sakelar fisik (*push button*) di *breadboard*, ATAU melalui tombol di antarmuka web *smartphone* Anda. Keduanya harus tersinkronisasi dua arah secara *real-time*. Jika Anda menekan tombol fisik di meja praktikum, status visual indikator di antarmuka *smartphone* harus otomatis berubah seketika, begitu pula sebaliknya jika dikontrol dari web.

Langkah Kerja Rangkaian:

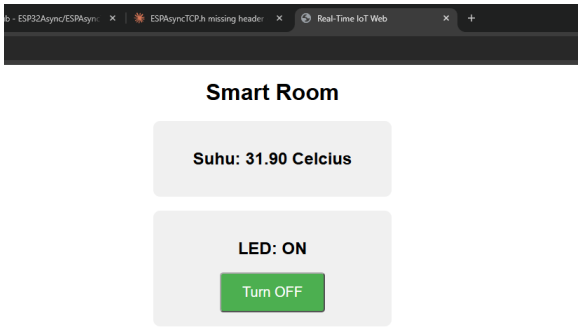
1. Hubungkan kaki Anoda (kaki panjang) LED ke pin D6 (GPIO 12). Hubungkan kaki Katoda (kaki pendek) LED ke salah satu kaki Resistor 220 Ohm, lalu hubungkan kaki resistor lainnya ke pin GND NodeMCU *(sama seperti rangkaian LED pada Modul 3)*.
2. Hubungkan pin DATA / OUT DHT ke pin D4 (GPIO 2). (VCC DHT ke 3V3, GND ke GND).
3. Pasang Push Button pada *breadboard*, melintasi parit tengah. Hubungkan salah satu kakinya ke jalur 3V3 (3.3V). Hubungkan kaki seberang tombol ke pin D2 (GPIO 4) NodeMCU. Pada titik kaki yang terhubung ke pin D2 tersebut, pasang salah satu kaki Resistor 10k Ohm, lalu hubungkan ujung kaki resistor lainnya ke jalur GND *(konfigurasi resistor pull-down eksternal seperti pada Modul 1)*.

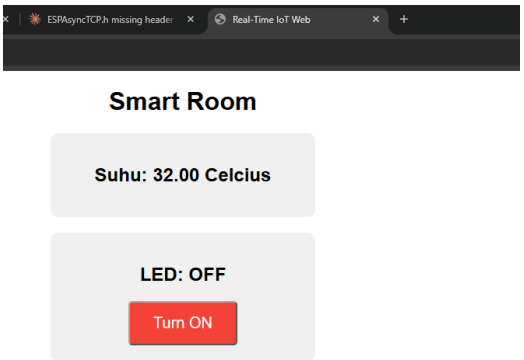
Langkah Kerja Pemrograman: Kode program ini cukup kompleks karena mengombinasikan *Front-End* (HTML, CSS, JavaScript) dengan *Back-End* (C++). Agar *real-time response* tombol tidak terganggu oleh sensor DHT yang butuh jeda baca lama, kita menerapkan teknik pemrograman *non-blocking* menggunakan fungsi millis().

Rangkaian



Hasil Pengujian





Latihan

Latihan 1: WebSockets vs Meta Refresh

Meta Refresh membuat browser memuat ulang seluruh halaman setiap interval tertentu. Setiap refresh berarti browser mengirim request HTTP baru lengkap dengan header, lalu ESP8266 mengirim ulang seluruh HTML, CSS, dan JavaScript. Padahal yang berubah mungkin hanya satu angka suhu. Karena halaman dimuat ulang dari nol, layar berkedip (flicker) dan seluruh tampilan digambar ulang. WebSockets hanya membuka koneksi satu kali (handshake di awal), lalu koneksi itu tetap terbuka dua arah. Setelah itu server cukup mengirim paket kecil berisi data saja, misalnya {"led":"1","suhu":"28.5"}, dan hanya saat ada perubahan (model *push*). Halaman HTML tidak dimuat ulang, JavaScript hanya mengganti teks pada elemen tertentu, sehingga layar tidak berkedip**.**

Latihan 2: Bahaya delay(3000) di loop()

Fungsi delay(3000) membuat NodeMCU berhenti total selama 3 detik, dan selama itu program tidak bisa mengerjakan apa pun. Padahal dalam sistem WebSockets, loop() harus terus berputar cepat untuk memeriksa tombol fisik, membersihkan koneksi klien lewat ws.cleanupClients(), dan memberi waktu bagi CPU untuk mengurus WiFi. Kalau program tertahan, tombol fisik yang ditekan saat jeda bisa terlewat, dan perintah dari browser baru dijalankan setelah jeda selesai, sehingga LED terasa lag dan tidak real-time lagi. Koneksi klien yang sudah putus juga menumpuk di memori. Yang paling berbahaya, ESP8266 punya watchdog timer. Kalau loop() memblokir terlalu lama, koneksi WiFi bisa putus atau board restart sendiri. Karena itu dipakai timer non-blocking dengan millis(), supaya program tetap berputar cepat dan sensor DHT hanya dibaca saat selisih waktunya sudah melewati 3000 ms.

Latihan 3: Target Injeksi pada JavaScript

Blok // Merombak UI Tombol & Teks sesuai status hardware bekerja saat server mengirim data led. Script memakai dua elemen HTML sebagai target. Yang pertama adalah ledStatus, yaitu yang teksnya diubah menjadi "ON" atau "OFF". Yang kedua adalah toggleBtn, yaitu tombol yang teksnya diganti menjadi "Turn OFF" atau "Turn ON", dan class CSS-nya diganti btn-on atau btn-off sehingga warnanya ikut berubah antara hijau dan merah. Fungsi onMessage() secara keseluruhan juga mengubah satu elemen lagi di luar blok tersebut, yaitu tempValue, yang diisi angka suhu dari dataObj.suhu. Jadi seluruh ID yang menjadi target injeksi teks adalah tempValue, ledStatus, dan toggleBtn.

Latihan 4 (Pengembangan Telemetri Sensor - Kelembapan):

Bagian kode yang di ubah

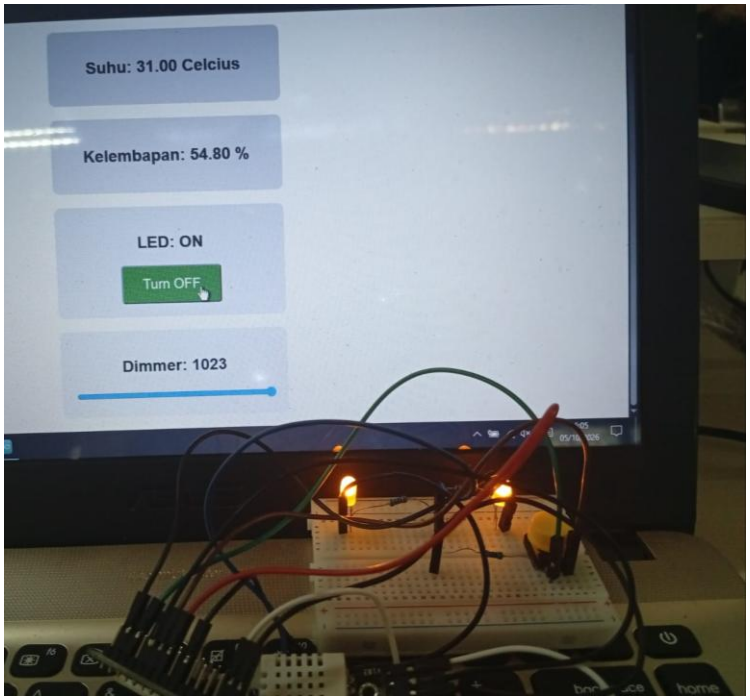
```
// Variabel Pelacak Status (State & Cache)
bool ledState = false;
String currentTemp = "--";
String currentHum = "--";      // BARU: penampung kelembapan
int buttonState;
int lastButtonState = LOW;
unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay = 50;
unsigned long lastTime = 0;
```

```
</head>
<body>
  <h1>Smart Room</h1>
  <div class="card">
    <h2>Suhu: <span id="tempValue">--</span> Celcius</h2>
  </div>
  <!-- BARU: kartu kelembapan -->
  <div class="card">
    <h2>Kelembapan: <span id="humValue">--</span> %</h2>
  </div>
  <div class="card">
    <h2>LED: <span id="ledStatus">OFF</span></h2>
    <button id="toggleBtn" class="btn-off" onclick="toggleLed()">Turn ON</button>
  </div>
```

```
// 3. Baca DHT setiap 3 detik (non-blocking)
if ((millis() - lastTime) > 3000) {
  float t = dht.readTemperature();
  float h = dht.readHumidity();          // BARU

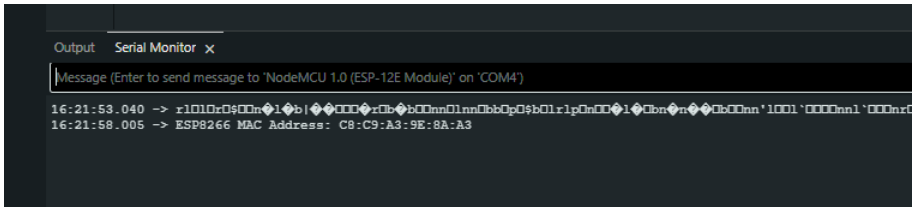
  if (!isnan(t) && !isnan(h)) {          // DIUBAH: cek keduanya valid
    currentTemp = String(t);
    currentHum  = String(h);             // BARU
    notifyClients();
  }
  lastTime = millis();
}
```

Hasil Penujian



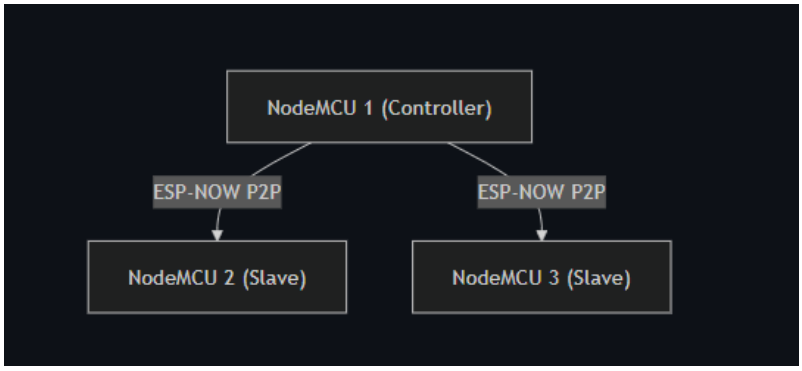
5.5.1 Praktikum 1: Identifikasi MAC Address Perangkat NodeMCU

Sebelum membangun jaringan ESP-NOW, setiap anggota kelompok wajib membaca dan mencatat alamat fisik (*MAC Address*) dari masing-masing *board* NodeMCU.



5.5.2 Praktikum 2: Komunikasi Nirkabel One-to-Many (ESP-NOW)

Skenario: Satu unit NodeMCU bertindak sebagai Controller / Sender, menyiarkan paket data instruksi yang sama secara langsung ke dua unit NodeMCU lain yang bertindak sebagai Slave / Receiver tanpa perantara router.



Langkah Kerja Pemrograman Sisi Pengirim (Controller):

1. Masukkan alamat MAC milik Board Penerima 1 dan Board Penerima 2 ke dalam variabel receiver1 dan receiver2.
2. Format tanda titik dua (:) diubah menjadi bilangan heksadesimal berawalan 0x yang dipisahkan oleh tanda koma (contoh: BC:FF:4D:23:45:90 →{0xBC, 0xFF, 0x4D, 0x23, 0x45, 0x90}).
3. Tuliskan kode berikut dan unggah ke NodeMCU 1:

```ino
#include <ESP8266WiFi.h>
#include <espnow.h>

// GANTI dengan MAC Address Board Penerima 1 dan Board Penerima 2 Anda!
uint8_t receiver1[] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};
uint8_t receiver2[] = {0xAA, 0xBB, 0xCC, 0xDD, 0xEE, 0xFF};

// Definisi struktur data paket instruksi
typedef struct struct_pesan {
  int perintahId;
  int nilaiParameter;
} struct_pesan;

struct_pesan paketKirim;

unsigned long previousMillis = 0;
const long interval = 2000; // Transmisi berkala setiap 2 detik

// Callback otomatis saat paket selesai dipancarkan radio
void OnDataSent(uint8_t *mac_addr, uint8_t sendStatus) {
  char macStr[18];
  snprintf(macStr, sizeof(macStr), "%02x:%02x:%02x:%02x:%02x:%02x",
           mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3], mac_addr[4], mac_addr[5]);

  Serial.print("Kirim paket ke: ");
  Serial.print(macStr);
  Serial.print(" | Status: ");
  if (sendStatus == 0) {
    Serial.println("Berhasil Diterima");
  } else {
    Serial.println("Gagal (Tidak Terjangkau)");
  }
}

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  WiFi.disconnect();

  // Inisialisasi stack protokol ESP-NOW
  if (esp_now_init() != 0) {
    Serial.println("Gagal menginisialisasi ESP-NOW!");
    return;
  }

  // Tetapkan peran board sebagai Controller
  esp_now_set_self_role(ESP_NOW_ROLE_CONTROLLER);
  esp_now_register_send_cb(OnDataSent);

  // Daftarkan kedua board penerima ke tabel peer internal
  esp_now_add_peer(receiver1, ESP_NOW_ROLE_SLAVE, 1, NULL, 0);
  esp_now_add_peer(receiver2, ESP_NOW_ROLE_SLAVE, 1, NULL, 0);

  Serial.println("Controller ESP-NOW Siap!");
}

void loop() {
  unsigned long currentMillis = millis();
  if (currentMillis - previousMillis >= interval) {
    previousMillis = currentMillis;

    paketKirim.perintahId = 101;
    paketKirim.nilaiParameter = random(10, 100);

    // Argumen 0 berarti mengirimkan paket ke seluruh peer yang terdaftar
    esp_now_send(0, (uint8_t *) &paketKirim, sizeof(paketKirim));
  }
}
```

Langkah Kerja Pemrograman Sisi Penerima (Slave):

1. Unggah kode sketsa berikut ke NodeMCU 2 dan NodeMCU 3.
2. Board penerima tidak perlu mengetahui alamat MAC milik Controller.

```ino
#include <ESP8266WiFi.h>
#include <espnow.h>

// Struktur data penerima (wajib identik dengan sisi pengirim)
typedef struct struct_pesan {
  int perintahId;
  int nilaiParameter;
} struct_pesan;

struct_pesan paketTerima;

// Callback otomatis saat antena mendeteksi paket masuk
void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
  char macStr[18];
  snprintf(macStr, sizeof(macStr), "%02x:%02x:%02x:%02x:%02x:%02x",
           mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3], mac_addr[4], mac_addr[5]);

  // Salin buffer memori radio langsung ke struktur data variabel
  memcpy(&paketTerima, incomingData, sizeof(paketTerima));

  Serial.print("Paket masuk dari: ");
  Serial.println(macStr);
  Serial.print("Ukuran Data: ");
  Serial.print(len);
  Serial.println(" byte");
  Serial.print("Perintah ID: ");
  Serial.println(paketTerima.perintahId);
  Serial.print("Nilai Parameter: ");
  Serial.println(paketTerima.nilaiParameter);
  Serial.println("-----------------------------------");
}

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  WiFi.disconnect();

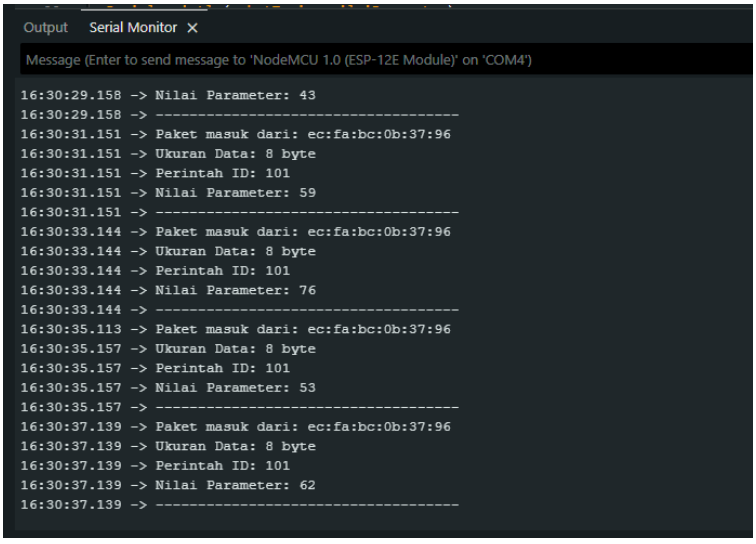
  if (esp_now_init() != 0) {
    Serial.println("Inisialisasi ESP-NOW Gagal!");
    return;
  }

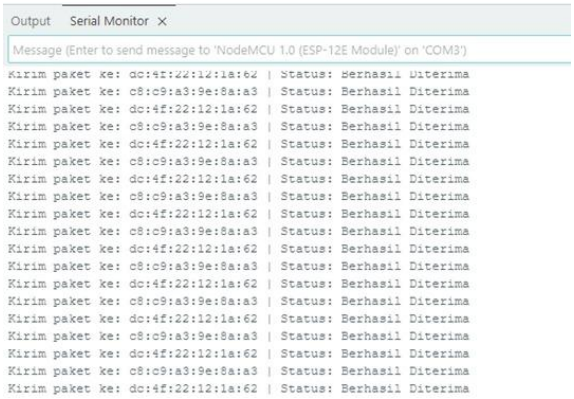
  // Tetapkan peran board sebagai Slave
  esp_now_set_self_role(ESP_NOW_ROLE_SLAVE);
  esp_now_register_recv_cb(OnDataRecv);

  Serial.println("Receiver ESP-NOW Menunggu Paket...");
}

void loop() {
  // Loop kosong: penerimaan data sepenuhnya ditangani oleh interrupt callback
}
```

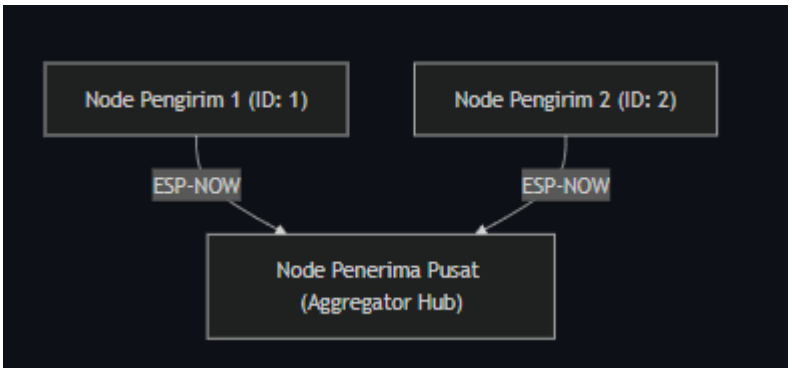
Hasil Pengujian: Sebagai NodeMCU 2





5.5.3 Praktikum 3: Pengumpulan Data Telemetri Many-to-One (ESP-NOW)

Skenario: Dua node sensor di lapangan memancarkan data pengukuran masing-masing ke sebuah node konsentrator pusat (*Aggregator Hub / Base Station*).



Langkah Kerja Pemrograman Sisi Node Pengirim:

1. Masukkan alamat MAC milik Board Penerima Pusat (Hub) pada variabel hubMacAddress.
2. Pada Board Pengirim 1, atur: #define BOARD_ID 1.
3. Pada Board Pengirim 2, atur: #define BOARD_ID 2.

```ino
#include <ESP8266WiFi.h>
#include <espnow.h>

// GANTI dengan MAC Address milik BOARD PENERIMA PUSAT (HUB)!
uint8_t hubMacAddress[] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};

// Identitas unik node (Ubah angka ini untuk node yang berbeda!)
#define BOARD_ID 1

typedef struct struct_sensor {
  int id;
  float pembacaan1;
  float pembacaan2;
} struct_sensor;

struct_sensor dataKirim;

unsigned long prevTime = 0;
const unsigned long sendInterval = 3000; // Kirim data setiap 3 detik

void OnDataSent(uint8_t *mac_addr, uint8_t sendStatus) {
  Serial.print("Pengiriman Node #");
  Serial.print(BOARD_ID);
  Serial.println(sendStatus == 0 ? " -> Sukses Diterima Hub" : " -> Gagal Sampai");
}

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  WiFi.disconnect();

  if (esp_now_init() != 0) {
    Serial.println("Inisialisasi ESP-NOW gagal");
    return;
  }

  esp_now_set_self_role(ESP_NOW_ROLE_CONTROLLER);
  esp_now_register_send_cb(OnDataSent);

  // Daftarkan hub penerima tunggal
  esp_now_add_peer(hubMacAddress, ESP_NOW_ROLE_SLAVE, 1, NULL, 0);
}

void loop() {
  unsigned long now = millis();
  if (now - prevTime >= sendInterval) {
    prevTime = now;

    dataKirim.id = BOARD_ID;
    dataKirim.pembacaan1 = random(250, 350) / 10.0; // Simulasi pembacaan suhu: 25.0 - 35.0 C
    dataKirim.pembacaan2 = random(400, 800) / 10.0; // Simulasi pembacaan kelembapan: 40.0 - 80.0 %

    esp_now_send(hubMacAddress, (uint8_t *) &dataKirim, sizeof(dataKirim));
  }
}
```

Langkah Kerja Pemrograman Sisi Hub Konsentrator:

Unggah kode sketsa berikut ke Board Penerima Pusat:

```ino
#include <ESP8266WiFi.h>
#include <espnow.h>

typedef struct struct_sensor {
  int id;
  float pembacaan1;
  float pembacaan2;
} struct_sensor;

struct_sensor dataTerima;

// Array penampung data terakhir: Index 0 untuk Node 1, Index 1 untuk Node 2
struct_sensor records[2];

void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
  memcpy(&dataTerima, incomingData, sizeof(dataTerima));

  // Validasi ID agar tidak terjadi error index array out-of-bounds
  if (dataTerima.id >= 1 && dataTerima.id <= 2) {
    int index = dataTerima.id - 1;
    records[index].id = dataTerima.id;
    records[index].pembacaan1 = dataTerima.pembacaan1;
    records[index].pembacaan2 = dataTerima.pembacaan2;

    Serial.println("=======================================");
    Serial.printf("DATA TERBARU DARI NODE #%d\n", dataTerima.id);
    Serial.printf("Parameter 1 : %.2f\n", records[index].pembacaan1);
    Serial.printf("Parameter 2 : %.2f\n", records[index].pembacaan2);
    Serial.println("=======================================");
  }
}

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  WiFi.disconnect();

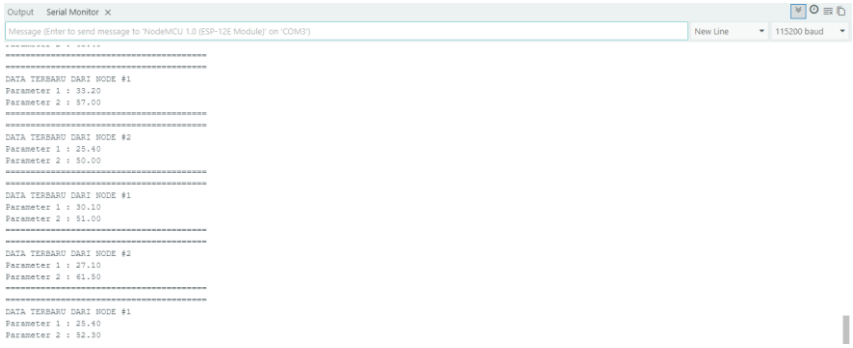
  if (esp_now_init() != 0) {
    Serial.println("Gagal Inisialisasi ESP-NOW Hub!");
    return;
  }

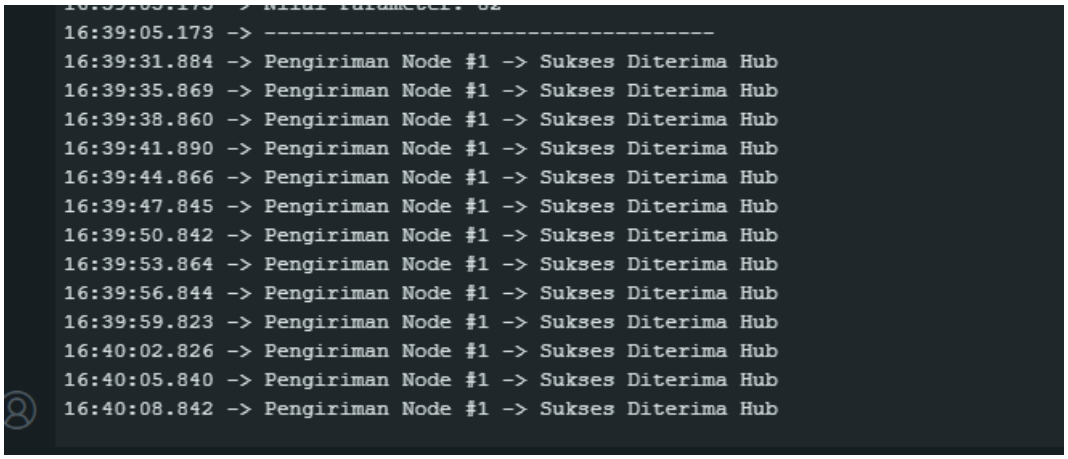
  esp_now_set_self_role(ESP_NOW_ROLE_SLAVE);
  esp_now_register_recv_cb(OnDataRecv);

  Serial.println("Hub Konsentrator Siap Menerima Data Sensor...");
}

void loop() {
  // Loop bebas menangani tugas lain
}
```

Hasil Pengujian:





5.5.4 Praktikum 4: Jaringan Sensor Ad-Hoc Multi-Hop (painlessMesh)

Skenario: Seluruh board dalam kelompok dihubungkan ke sensor suhu & kelembapan riil (DHT11/DHT22) dan diintegrasikan ke dalam sebuah jaringan rantai otonom (*Mesh*). Setiap board secara berkala membaca sensor fisiknya, mengemas data telemetri ke dalam format **JSON**, dan menyiarkannya ke seluruh anggota jaringan tanpa perlu mengetahui alamat MAC masing-masing.

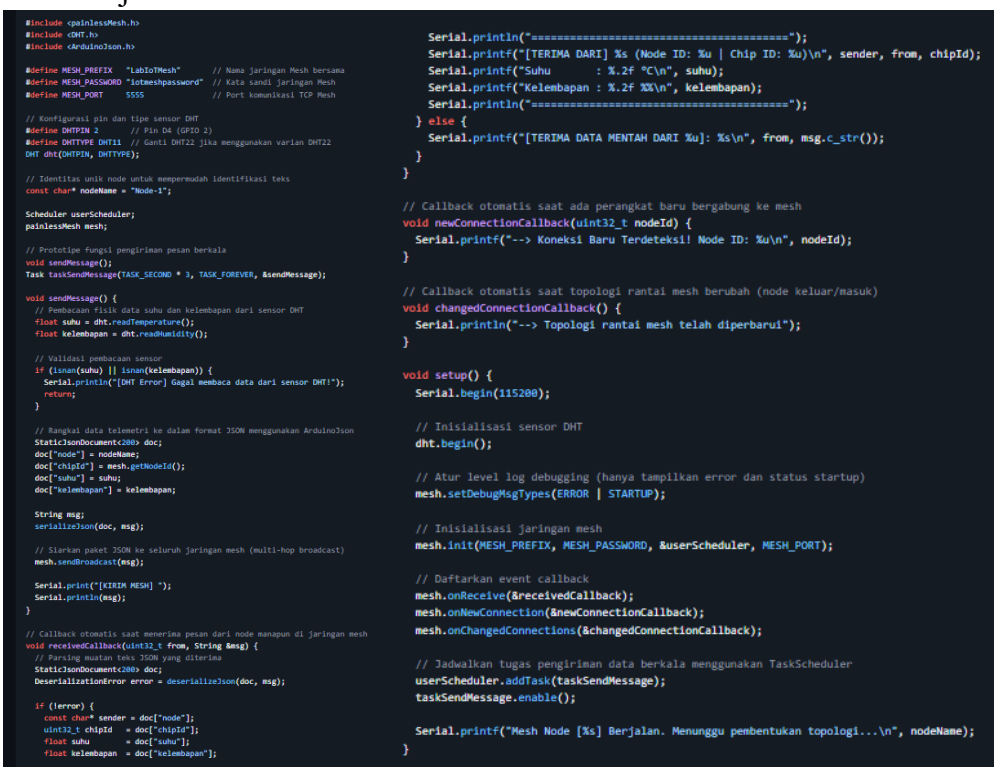
Instruksi Perangkaian Perangkat Keras:

- Pasang sensor DHT11 / DHT22 pada masing-masing board NodeMCU:
    - Pin VCC sensor ke pin 3V3 (3.3V) NodeMCU.
    - Pin GND sensor ke pin GND NodeMCU.
    - Pin DATA / OUT sensor ke pin D4 (GPIO 2) NodeMCU.

Langkah Kerja Pemrograman:

1. Unggah sketsa berikut ke seluruh board dalam kelompok Anda.
2. Berikan nama pengenal unik pada variabel nodeName di masing-masing board (misal: "Node-1", "Node-2", "Node-3").

3. Jika menggunakan varian DHT22, ubah #define DHTTYPE DHT11 menjadi DHT22.



```
#include <painlessMesh.h>
#include <DHT.h>
#include <ArduinoJson.h>

#define MESH_PREFIX   "LabIoTMesh"      // Nama jaringan Mesh bersama
#define MESH_PASSWORD "iotmeshpassword" // Kata sandi jaringan Mesh
#define MESH_PORT     5555              // Port komunikasi TCP Mesh

// Konfigurasi pin dan tipe sensor DHT
#define DHTPIN 2      // Pin D4 (GPIO 2)
#define DHTTYPE DHT11  // Ganti DHT22 jika menggunakan varian DHT22
DHT dht(DHTPIN, DHTTYPE);

// Identitas unik node untuk mempermudah identifikasi teks
const char* nodeName = "Node-1";

Scheduler userScheduler;
painlessMesh mesh;

// Prototipe fungsi pengiriman pesan berkala
void sendMessage();
Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);

void sendMessage() {
  // Pembacaan fisik data suhu dan kelembapan dari sensor DHT
  float suhu = dht.readTemperature();
  float kelembapan = dht.readHumidity();

  // Validasi pembacaan sensor
  if (isnan(suhu) || isnan(kelembapan)) {
    Serial.println("[DHT Error] Gagal membaca data dari sensor DHT!");
    return;
  }

  // Rangkai data telemetri ke dalam format JSON menggunakan ArduinoJson
  StaticJsonDocument<200> doc;
  doc["node"] = nodeName;
  doc["chipId"] = mesh.getNodeId();
  doc["suhu"] = suhu;
  doc["kelembapan"] = kelembapan;

  String msg;
  serializeJson(doc, msg);

  // Siarkan paket JSON ke seluruh jaringan mesh (multi-hop broadcast)
  mesh.sendBroadcast(msg);

  Serial.print("[KIRIM MESH] ");
  Serial.println(msg);
}

// Callback otomatis saat menerima pesan dari node manapun di jaringan mesh
void receivedCallback(uint32_t from, String &msg) {
  // Parsing muatan teks JSON yang diterima
  StaticJsonDocument<200> doc;
  DeserializationError error = deserializeJson(doc, msg);

  if (!error) {
    const char* sender = doc["node"];
    uint32_t chipId    = doc["chipId"];
    float suhu         = doc["suhu"];
    float kelembapan   = doc["kelembapan"];

    Serial.println("=====================================");
    Serial.printf("[TERIMA DARI] %s (Node ID: %u | Chip ID: %u)\n", sender, from, chipId);
    Serial.printf("Suhu       : %.2f °C\n", suhu);
    Serial.printf("Kelembapan : %.2f %%\n", kelembapan);
    Serial.println("=====================================");
  } else {
    Serial.printf("[TERIMA DATA MENTAH DARI %u]: %s\n", from, msg.c_str());
  }
}

// Callback otomatis saat ada perangkat baru bergabung ke mesh
void newConnectionCallback(uint32_t nodeId) {
  Serial.printf("--> Koneksi Baru Terdeteksi! Node ID: %u\n", nodeId);
}

// Callback otomatis saat topologi rantai mesh berubah (node keluar/masuk)
void changedConnectionCallback() {
  Serial.println("--> Topologi rantai mesh telah diperbarui");
}

void setup() {
  Serial.begin(115200);

  // Inisialisasi sensor DHT
  dht.begin();

  // Atur level log debugging (hanya tampilkan error dan status startup)
  mesh.setDebugMsgTypes(ERROR | STARTUP);

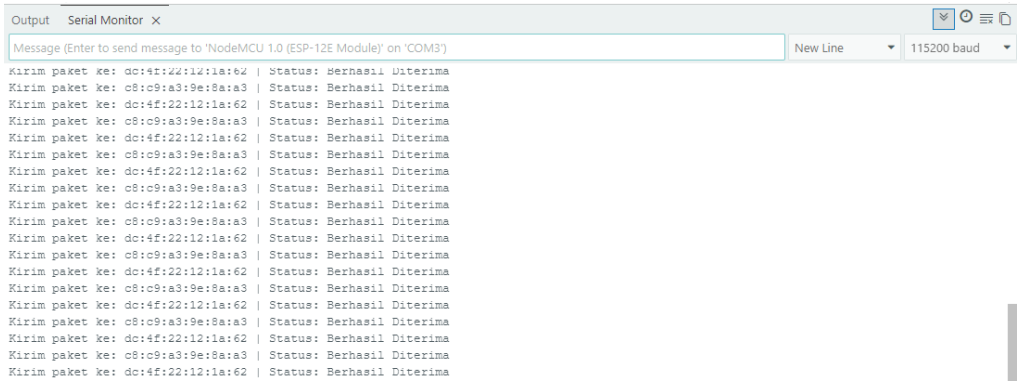
  // Inisialisasi jaringan mesh
  mesh.init(MESH_PREFIX, MESH_PASSWORD, &userScheduler, MESH_PORT);

  // Daftarkan event callback
  mesh.onReceive(&receivedCallback);
  mesh.onNewConnection(&newConnectionCallback);
  mesh.onChangedConnections(&changedConnectionCallback);

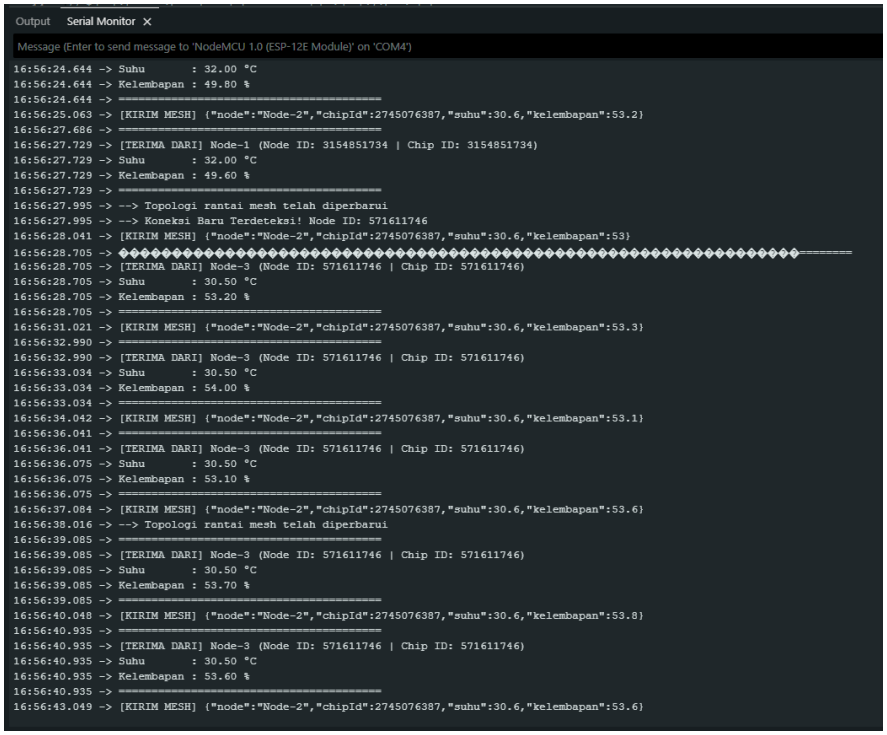
  // Jadwalkan tugas pengiriman data berkala menggunakan TaskScheduler
  userScheduler.addTask(taskSendMessage);
  taskSendMessage.enable();

  Serial.printf("Mesh Node [%s] Berjalan. Menunggu pembentukan topologi...\n", nodeName);
}
```

Hasil Pengujian:



```
Output  Serial Monitor ×
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')   New Line   115200 baud
Kirim paket ke: dc:4f:22:12:1a:62 | Status: Berhasil Diterima
Kirim paket ke: c8:c9:a3:9e:8a:a3 | Status: Berhasil Diterima
Kirim paket ke: dc:4f:22:12:1a:62 | Status: Berhasil Diterima
Kirim paket ke: c8:c9:a3:9e:8a:a3 | Status: Berhasil Diterima
Kirim paket ke: dc:4f:22:12:1a:62 | Status: Berhasil Diterima
Kirim paket ke: c8:c9:a3:9e:8a:a3 | Status: Berhasil Diterima
Kirim paket ke: dc:4f:22:12:1a:62 | Status: Berhasil Diterima
Kirim paket ke: c8:c9:a3:9e:8a:a3 | Status: Berhasil Diterima
Kirim paket ke: dc:4f:22:12:1a:62 | Status: Berhasil Diterima
Kirim paket ke: c8:c9:a3:9e:8a:a3 | Status: Berhasil Diterima
Kirim paket ke: dc:4f:22:12:1a:62 | Status: Berhasil Diterima
Kirim paket ke: c8:c9:a3:9e:8a:a3 | Status: Berhasil Diterima
Kirim paket ke: dc:4f:22:12:1a:62 | Status: Berhasil Diterima
Kirim paket ke: c8:c9:a3:9e:8a:a3 | Status: Berhasil Diterima
Kirim paket ke: dc:4f:22:12:1a:62 | Status: Berhasil Diterima
Kirim paket ke: c8:c9:a3:9e:8a:a3 | Status: Berhasil Diterima
Kirim paket ke: dc:4f:22:12:1a:62 | Status: Berhasil Diterima
```


Output Serial Monitor ×
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4')
16:56:24.644 -> Suhu : 32.00 °C
16:56:24.644 -> Kelembapan : 49.80 %
16:56:24.644 -> ==
16:56:25.063 -> [KIRIM MESH] {"node":"Node-2","chipId":2745076387,"suhu":30.6,"kelembapan":53.2}
16:56:27.686 -> ==
16:56:27.729 -> [TERIMA DARI] Node-1 (Node ID: 3154851734 | Chip ID: 3154851734)
16:56:27.729 -> Suhu : 32.00 °C
16:56:27.729 -> Kelembapan : 49.60 %
16:56:27.729 -> ==
16:56:27.995 -> --> Topologi rantai mesh telah diperbarui
16:56:27.995 -> --> Koneksi Baru Terdeteksi! Node ID: 571611746
16:56:28.041 -> [KIRIM MESH] {"node":"Node-2","chipId":2745076387,"suhu":30.6,"kelembapan":53}
16:56:28.705 -> ◆◆◆

Latihan 3 (Eksperimen Modifikasi Filter Telemetri & Ambang Batas JSON - Praktikum 4): Pada Praktikum 4, setiap node saat ini mencetak seluruh data telemetri dari semua pengirim tanpa penyaringan.

```
[SUHU KRITIS] Node-1 (Node ID: 3154851734 | Chip ID: 3154851734)
Suhu       : 37.30 °C
Kelembapan : 40.50 %
=======================================
[KIRIM MESH] {"node":"Node-3","chipId":571611746,"suhu":31.1,"kelembapan":53.5}
[NORMAL] Telemetri dari Node 2745076387 aman
=======================================
[SUHU KRITIS] Node-1 (Node ID: 3154851734 | Chip ID: 3154851734)
Suhu       : 37.50 °C
Kelembapan : 40.10 %
=======================================
[KIRIM MESH] {"node":"Node-3","chipId":571611746,"suhu":31.1,"kelembapan":53.1}
[NORMAL] Telemetri dari Node 2745076387 aman
=======================================
[SUHU KRITIS] Node-1 (Node ID: 3154851734 | Chip ID: 3154851734)
Suhu       : 37.50 °C
Kelembapan : 39.90 %
=======================================
[KIRIM MESH] {"node":"Node-3","chipId":571611746,"suhu":31.1,"kelembapan":52.7}
[NORMAL] Telemetri dari Node 2745076387 aman
```